

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Нейронные сети»
Тема: Распознавание объектов на фотографиях

Студент гр. 1306

Шаврин А.П.

Преподаватель

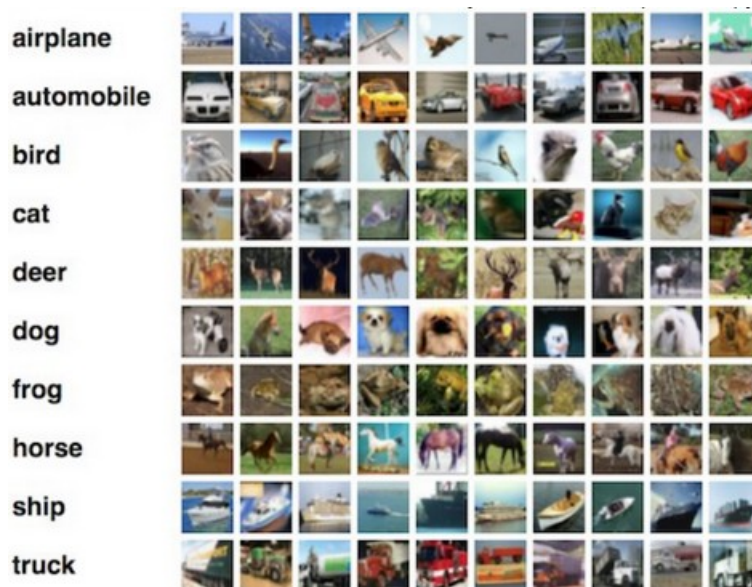
Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Распознавание объектов на фотографиях (Object Recognition in Photographs) CIFAR-10 (классификация небольших изображений по десяти классам: самолет, автомобиль, птица, кошка, олень, собака, лягушка, лошадь, корабль и грузовик).



Задание.

- Ознакомиться со сверточными нейронными сетями
- Изучить построение модели в Keras в функциональном виде
- Изучить работу слоя разреживания (Dropout)

Выполнение работы.

В работе используется функция `cifar10.load_data()` из Keras, которая автоматически загружает и разделяет данные. CIFAR-10 состоит из 60000 цветных изображений размером 32×32 пикселя, разделенных на 10 классов: самолет, автомобиль, птица, кошка, олень, собака, лягушка, лошадь, корабль, грузовик. Обучающая выборка содержит 50000 изображений, тестовая – 10000. Для загруженного датасета выполняется нормализация пикселей (деление на 255) для приведения значений к диапазону [0,1] и преобразование меток в one-hot векторы для использования с categorical crossentropy (рисунок 1).

Загрузка данных

```
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
print("Размер обучающей выборки:", X_train.shape)
print("Размер тестовой выборки:", X_test.shape)
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 6s 0us/step
```

```
Размер обучающей выборки: (50000, 32, 32, 3)
```

```
Размер тестовой выборки: (10000, 32, 32, 3)
```

Предобработка данных

```
# Нормализация
X_train = X_train.astype('float32') / 255.0
X_test  = X_test.astype('float32') / 255.0

# Преобразование меток в one-hot векторы
num_classes = len(np.unique(y_train))
Y_train = to_categorical(y_train, num_classes)
Y_test  = to_categorical(y_test, num_classes)
```

Рисунок 1. Чтение и предобработка данных.

Для проведения экспериментов была написана функция `create_cnn` (рисунок 2), позволяющая задавать архитектуру сети с определенными размерами ядра и дропаутом. Модель состоит из следующих блоков:

- Входной слой: изображения размером 32×32 с 3 каналами (RGB).
- Блок 1:
 - Сверточный слой (32 фильтра, ядро $n \times n$, padding='same', активация ReLU).
 - Сверточный слой (32 фильтра, ядро $n \times n$, padding='same', активация ReLU).
 - MaxPooling (пул 2×2).
 - Опциональный Dropout (0.25).
- Блок 2:
 - Сверточный слой (64 фильтра, ядро $n \times n$, padding='same', активация ReLU).

- Сверточный слой (64 фильтра, ядро $n \times n$, padding='same', активация ReLU).
- MaxPooling (пул 2×2).
- Опциональный Dropout (0.25).
- Классификационная часть:
 - Flatten – преобразование карт признаков в вектор.
 - Полносвязный слой (512 нейронов, активация ReLU).
 - Опциональный Dropout (0.5).
 - Выходной слой (10 нейронов, активация softmax).

На рис. 3 представлена схема прохождения данных через сеть.

Генерация модели

```
def create_cnn(kernel_size=3, use_dropout=True):
    """
    Создает сверточную нейронную сеть для CIFAR-10.

    Параметры:
        kernel_size : int – размер ядра свертки (по умолчанию 3)
        use_dropout : bool – использовать ли Dropout слои
    """
    # Входной слой: изображения 32x32 с 3 каналами (RGB)
    inp = Input(shape=(32, 32, 3))

    # Блок 1: две свертки 32 фильтра, затем подвыборка и опционально Dropout
    conv_1 = Conv2D(32, kernel_size, padding='same', activation='relu')(inp)
    conv_2 = Conv2D(32, kernel_size, padding='same', activation='relu')(conv_1)
    pool_1 = MaxPooling2D(pool_size=(2, 2))(conv_2)
    if use_dropout:
        pool_1 = Dropout(0.25)(pool_1)

    # Блок 2: две свертки 64 фильтра, затем подвыборка и опционально Dropout
    conv_3 = Conv2D(64, kernel_size, padding='same', activation='relu')(pool_1)
    conv_4 = Conv2D(64, kernel_size, padding='same', activation='relu')(conv_3)
    pool_2 = MaxPooling2D(pool_size=(2, 2))(conv_4)
    if use_dropout:
        pool_2 = Dropout(0.25)(pool_2)

    # Переход к полносвязным слоям
    flat = Flatten()(pool_2)
    hidden = Dense(512, activation='relu')(flat)
    if use_dropout:
        hidden = Dropout(0.5)(hidden)
    out = Dense(num_classes, activation='softmax')(hidden)

    model = Model(inputs=inp, outputs=out)
    return model
```

Рисунок 2. Функция создания модели.

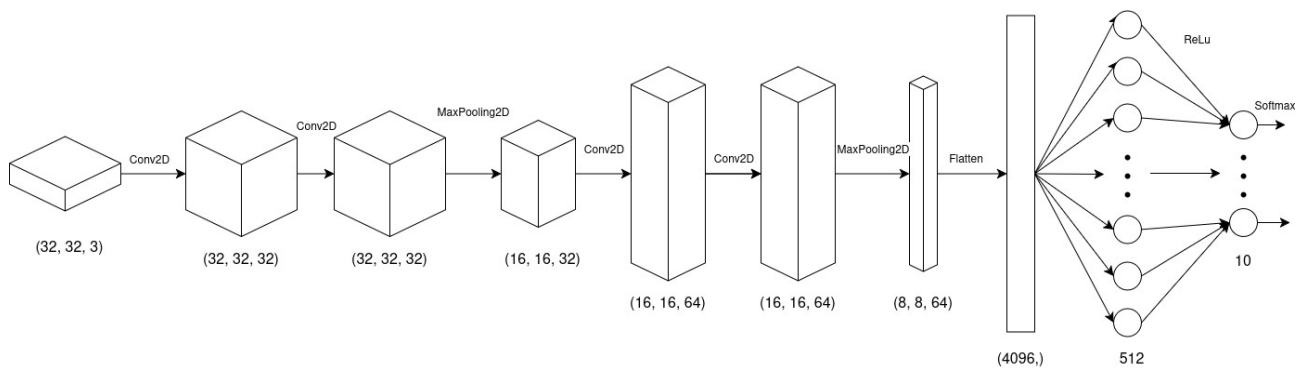


Рисунок 3. Схема модели.

Для обучения используется функция `train_and_evaluate` (рисунок 4). Модель компилируется с оптимизатором Adam, функцией потерь `categorical_crossentropy` и метрикой accuracy. Обучение проводится в течение 20 эпох (вместо 200, указанных в методичке) с размером батча 32 и валидационной выборкой 10% от обучающей. Выбор меньшего числа эпох обусловлен ограниченными вычислительными ресурсами.

Обучение и оценка модели

```
def train_and_evaluate(model, epochs=20, batch_size=32, verbose=1):
    model.compile(
        loss='categorical_crossentropy',
        optimizer=Adam(),
        metrics=['accuracy']
    )
    history = model.fit(
        X_train, Y_train,
        batch_size=batch_size,
        epochs=epochs,
        validation_split=0.1,
        verbose=verbose
    )
    score = model.evaluate(X_test, Y_test, verbose=0)
    print(f"Test loss: {score[0]:.4f}, Test accuracy: {score[1]:.4f}")
    return history, score[1]
```

Рисунок 4. Функция тренировки и оценки модели

Исследование влияния размера ядра и Dropout

Проведены эксперименты для двух значений размера ядра (3 и 5) и двух режимов использования Dropout (True/False). Всего обучено 4 модели. Графики представлены на рис. 5–7.

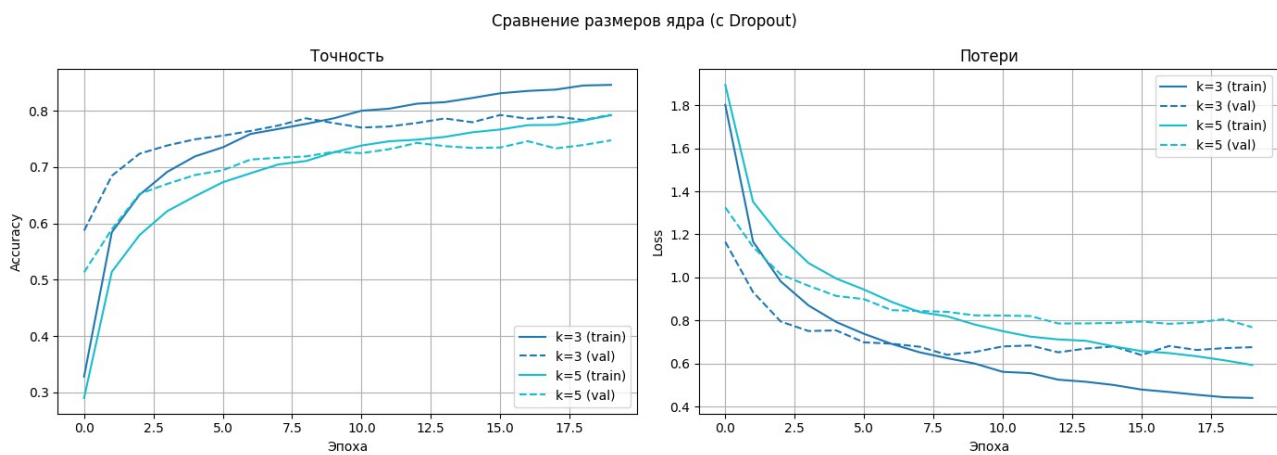


Рисунок 5. Графики точности и потерь в ходе обучения для разных размеров ядер с dropout.

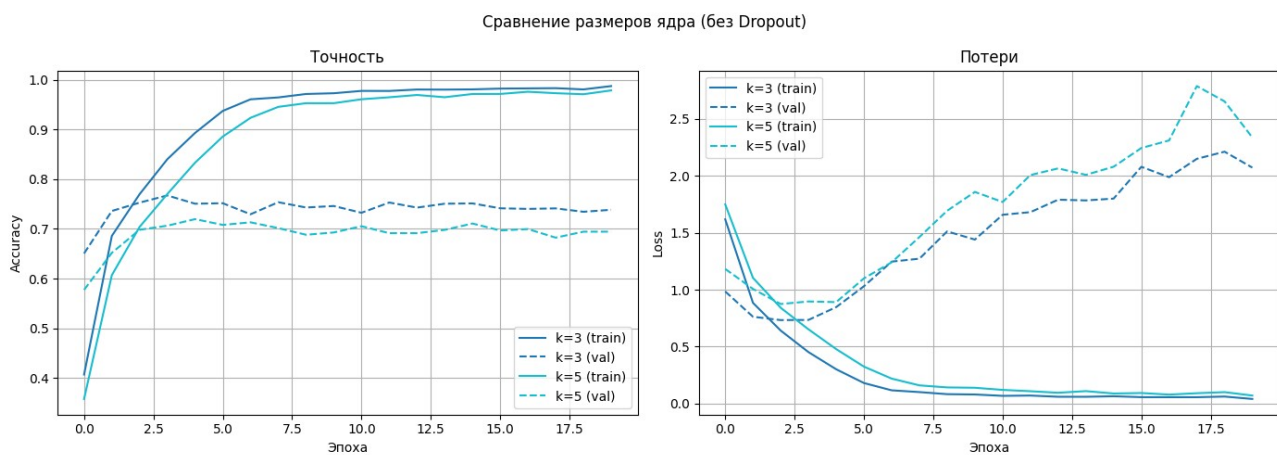


Рисунок 6. Графики точности и потерь в ходе обучения для разных размеров ядер без dropout.

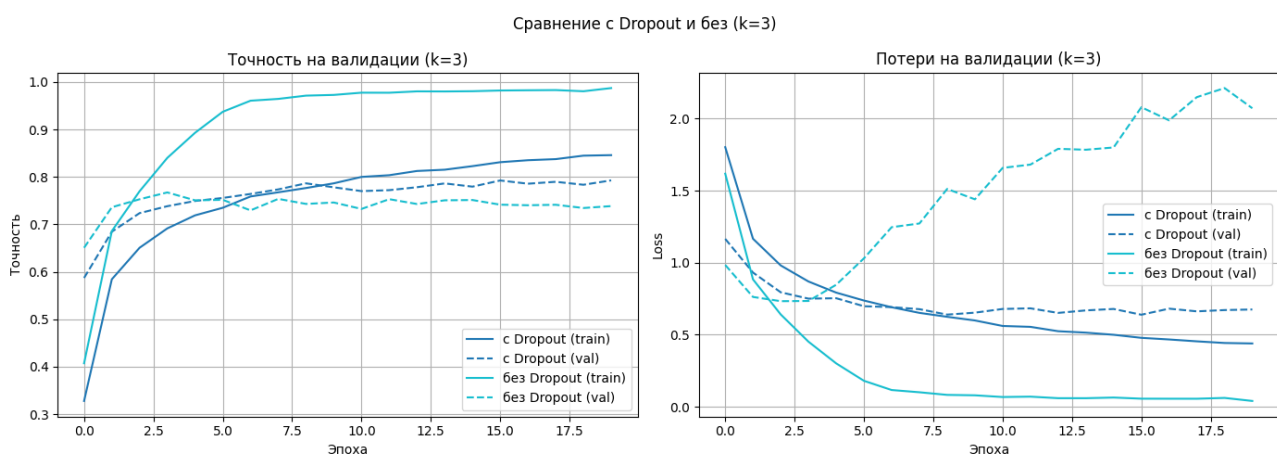


Рисунок 7. График точности и потерь в ходе обучения для ядра размером 3 с dropout и без.

Анализируя полученные графики, можно сделать следующие наблюдения:

- На графике 6 видно, что без использования Dropout кривая точности на обучении быстро достигает значений близких к 1.0, в то время как точность на валидации останавливается на уровне около 70-75%. Это классический признак переобучения – сеть запоминает обучающие примеры, но плохо обобщает.
- С Dropout обучение происходит медленнее (рисунок 5), но валидационная точность оказывается выше (до 79% для $k=3$). Потери на валидации при использовании Dropout стабилизируются на более низком уровне, что подтверждает эффективность регуляризации.
- Сравнение моделей с одинаковым режимом Dropout (рисунок 6), но разным размером ядра показывает, что ядро 3×3 работает лучше, чем 5×5 . Для изображений размером 32×32 ядро 5×5 захватывает слишком большую область, что может приводить к потере мелких деталей и ухудшению обобщающей способности. Кроме того, большее ядро увеличивает количество параметров и риск переобучения. Тестовая точность для $k=3$ с Dropout составляет 77.86%, для $k=5$ с Dropout – 73.92%; без Dropout разница также заметна (72.62% против 68.38%).

Выводы.

В ходе лабораторной работы была реализована сверточная нейронная сеть для классификации изображений CIFAR-10. Проведено исследование влияния Dropout и размера ядра свертки. Эксперименты подтвердили теоретические положения. Dropout является эффективным средством борьбы с переобучением в сверточных сетях, особенно при ограниченном размере выборки. Для набора CIFAR-10 оптимальным оказался размер ядра 3×3 ; увеличение ядра до 5×5 ухудшает результаты. Наилучшая тестовая точность (77.86%) достигнута моделью с `kernel_size=3` и использованием Dropout. Даже при сокращении числа эпох до 20 (вместо 200) все тенденции прослеживаются отчетливо, что позволяет сделать корректные выводы без излишних временных затрат (обучение всех моделей заняло более 10 часов).

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab5.ipynb

```
"""# Установка зависимостей
"""
```

```
!pip install pandas numpy matplotlib scikit-learn tensorflow
```

```
"""# Импорт"""
```

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Dense, Dropout,
```

Flatten

```
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
```

```
"""# Загрузка данных"""
```

```
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
print("Размер обучающей выборки:", X_train.shape)
print("Размер тестовой выборки:", X_test.shape)
```

```
"""# Предобработка данных"""
```

```
# Нормализация
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
```

```
# Преобразование меток в one-hot векторы
num_classes = len(np.unique(y_train))
Y_train = to_categorical(y_train, num_classes)
Y_test = to_categorical(y_test, num_classes)
```

```
"""# Генерация модели
```

```
"""
```

```
def create_cnn(kernel_size=3, use_dropout=True):
    """
```

Создает сверточную нейронную сеть для CIFAR-10.

Параметры:

```
    kernel_size : int – размер ядра свертки (по умолчанию 3)
    use_dropout : bool – использовать ли Dropout слои
    """
```

```
# Входной слой: изображения 32x32 с 3 каналами (RGB)
inp = Input(shape=(32, 32, 3))
```



```

# Блок 1: две свертки 32 фильтра, затем подвыборка и опционально Dropout
conv_1 = Conv2D(32, kernel_size, padding='same', activation='relu')(inp)
conv_2 = Conv2D(32, kernel_size, padding='same', activation='relu')(conv_1)
pool_1 = MaxPooling2D(pool_size=(2, 2))(conv_2)
if use_dropout:
    pool_1 = Dropout(0.25)(pool_1)

# Блок 2: две свертки 64 фильтра, затем подвыборка и опционально Dropout
conv_3 = Conv2D(64, kernel_size, padding='same', activation='relu')(pool_1)
conv_4 = Conv2D(64, kernel_size, padding='same', activation='relu')(conv_3)
pool_2 = MaxPooling2D(pool_size=(2, 2))(conv_4)
if use_dropout:
    pool_2 = Dropout(0.25)(pool_2)

# Переход к полносвязным слоям
flat = Flatten()(pool_2)
hidden = Dense(512, activation='relu')(flat)
if use_dropout:
    hidden = Dropout(0.5)(hidden)
out = Dense(num_classes, activation='softmax')(hidden)

model = Model(inputs=inp, outputs=out)
return model

"""# Обучение и оценка модели"""

def train_and_evaluate(model, epochs=20, batch_size=32, verbose=1):
    model.compile(
        loss='categorical_crossentropy',
        optimizer=Adam(),
        metrics=['accuracy']
    )
    history = model.fit(
        X_train, Y_train,
        batch_size=batch_size,
        epochs=epochs,
        validation_split=0.1,
        verbose=verbose
    )
    score = model.evaluate(X_test, Y_test, verbose=0)
    print(f"Test loss: {score[0]:.4f}, Test accuracy: {score[1]:.4f}")
    return history, score[1]

"""# Исследование"""

kernel_sizes = [3, 5]
results = {}
histories = {}
for k in kernel_sizes:
    for dropout in [True, False]:
        name = f"Dropout={dropout}, k={k}"

```

```

print(name)
model = create_cnn(kernel_size=k, use_dropout=dropout)
hist, acc = train_and_evaluate(model, epochs=20, verbose=1)
results[(dropout, k)] = (hist, acc)
histories[(dropout, k)] = hist

def plot_dropout_group(histories, kernel_sizes, dropout):
    plt.figure(figsize=(14, 5))
    plt.suptitle(f"Сравнение размеров ядра ({'с' if dropout else 'без'} Dropout)")

    colors = plt.cm.tab10(np.linspace(0, 1, len(kernel_sizes)))

    plt.subplot(1, 2, 1)
    for idx, k in enumerate(kernel_sizes):
        hist = histories[(dropout, k)]
        color = colors[idx]
        plt.plot(hist.history['accuracy'], label=f'k={k} (train)', color=color)
        plt.plot(hist.history['val_accuracy'], '--', label=f'k={k} (val)', color=color)
    plt.title(f"Точность")
    plt.xlabel("Эпоха")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.grid(True)

    plt.subplot(1, 2, 2)
    for idx, k in enumerate(kernel_sizes):
        hist = histories[(dropout, k)]
        color = colors[idx]
        plt.plot(hist.history['loss'], label=f'k={k} (train)', color=color)
        plt.plot(hist.history['val_loss'], '--', label=f'k={k} (val)', color=color)
    plt.title("Потери")
    plt.xlabel("Эпоха")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()

def plot_kernel_size(histories, kernel_size):
    plt.figure(figsize=(14, 5))
    plt.suptitle(f"Сравнение с Dropout и без (k={kernel_size})")

    colors = plt.cm.tab10(np.linspace(0, 1, len(kernel_sizes)))

    plt.subplot(1, 2, 1)
    hist_drop = histories[(True, kernel_size)]
    plt.plot(hist_drop.history['accuracy'], label='с Dropout (train)', color=colors[0])
    plt.plot(hist_drop.history['val_accuracy'], '--', label='с Dropout (val)', color=colors[0])
    hist_nodrop = histories[(False, kernel_size)]
    plt.plot(hist_nodrop.history['accuracy'], label='без Dropout (train)', color=colors[1])
    plt.plot(hist_nodrop.history['val_accuracy'], '--', label='без Dropout (val)', color=colors[1])

```

```

plt.title(f'Точность на валидации (k={kernel_size})')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(hist_drop.history['loss'], label='с Dropout (train)', color=colors[0])
plt.plot(hist_drop.history['val_loss'], '--', label='с Dropout (val)', color=colors[0])
plt.plot(hist_nodrop.history['loss'], label='без Dropout (train)', color=colors[1])
plt.plot(hist_nodrop.history['val_loss'], '--', label='без Dropout (val)', color=colors[1])
plt.title(f'Потери на валидации (k={kernel_size})')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

plot_dropout_group(histories, kernel_sizes, True)

plot_dropout_group(histories, kernel_sizes, False)

plot_kernel_size(histories, 3)

```